

Caso práctico 1 — UD 5

Validación de formularios web y documentación con JavaScript

Índice

1. Introducción
2. Objetivos de la práctica
3. Estructura del proyecto
4. Diseño del formulario HTML y validación básica con HTML5
5. Validación con JavaScript y expresiones regulares
6. Gestión de eventos y experiencia de usuario
7. Pruebas unitarias con Jest y documentación con JSDoc
8. Conclusión
9. Referencias

1. Introducción

En esta práctica he desarrollado un formulario de inscripción para cursos online con validación en el lado del cliente, utilizando HTML5 y JavaScript. La intención no ha sido solo crear una interfaz funcional, sino también comprobar que los datos introducidos sean correctos antes de permitir el envío del formulario.

Para conseguirlo, he combinado la validación básica de HTML5 con comprobaciones más completas en JavaScript, apoyándome en expresiones regulares, validación en tiempo real y gestión de eventos. También se han añadido mensajes de error y señales visuales para que la persona usuaria pueda detectar y corregir los fallos de forma más clara durante la interacción con la página.

Además, el proyecto incluye pruebas unitarias con Jest para verificar el comportamiento de las funciones de validación y documentación con JSDoc para describir el código de forma ordenada. En conjunto, esta práctica me ha permitido trabajar no solo la parte visual del formulario, sino también aspectos relacionados con la lógica, la usabilidad y la calidad del código.

2. Objetivos de la práctica

El objetivo principal ha sido desarrollar un formulario web capaz de validar correctamente la información introducida antes de su envío, utilizando para ello herramientas propias del desarrollo en entorno cliente. A través de este ejercicio se busca no solo construir una página funcional, sino también aplicar una validación más completa y controlada desde JavaScript.

Entre los objetivos más concretos, se encuentran la creación de un formulario estructurado en HTML, el uso de validaciones básicas con HTML5, la aplicación de expresiones regulares para comprobar campos concretos y la gestión de eventos como *input*, *focus*, *blur* y *submit*. También se pretende mejorar la experiencia de uso mostrando mensajes de error claros y retroalimentación visual en tiempo real.

Otro objetivo importante ha sido separar la lógica de validación en funciones reutilizables, de forma que el código resulte más claro y fácil de mantener. A esto se suma la incorporación de pruebas unitarias con Jest, que permiten comprobar si las funciones responden correctamente ante distintos casos, y la documentación con JSDoc, útil para describir el funcionamiento del código de una forma más ordenada.

En conjunto, la práctica está orientada a aplicar buenas bases de validación, organización y comprobación del código dentro de un caso realista de desarrollo frontend.

3. Estructura del proyecto

Para organizar mejor el trabajo, se ha optado por dividir los distintos elementos en varios archivos, separando la estructura, el estilo, la lógica de validación y las pruebas. Esta distribución hace que cada parte tenga una función concreta y facilita tanto la revisión como el mantenimiento del código.

El archivo `index.html` recoge la estructura principal del formulario y los campos que debe completar la persona usuaria. En él también se incluyen algunos atributos de validación básica de HTML5, como los campos obligatorios o el tipo de dato asociado al correo electrónico.

El archivo `styles.css` se encarga del aspecto visual. Aquí se han definido tanto el diseño general del formulario como los estilos relacionados con los estados de error, foco y validación correcta, de manera que la interacción resulte más clara a nivel visual.

La lógica de validación se reparte entre `script.js` y `validadores.js`. En `script.js` se controla el comportamiento del formulario, los eventos y la validación en tiempo real, mientras que en `validadores.js` se agrupan las funciones reutilizables encargadas de comprobar cada campo. Esta separación evita concentrar toda la lógica en un único archivo y hace que el código resulte más claro.

Además, se incluye el archivo `validadores.test.js`, donde se han desarrollado las pruebas unitarias con Jest para comprobar distintos casos válidos e inválidos. Junto a ello, la carpeta `docs` recoge la documentación generada con JSDoc a partir de los comentarios incluidos en el código.

De esta forma, cada archivo queda centrado en una responsabilidad concreta, lo que ayuda a mantener una organización más clara y coherente.

 docs	07/03/2026 18:04	File folder
 img	07/03/2026 19:07	File folder
 index	07/03/2026 19:43	HTML Source File
 package	07/03/2026 20:16	JSON Source File
 package-lock	07/03/2026 20:53	JSON Source File
 README	07/03/2026 20:52	Markdown Source...
 script	07/03/2026 19:47	JavaScript Source ...
 styles	07/03/2026 19:12	CSS Source File
 validadores	07/03/2026 21:45	JavaScript Source ...
 validadores.test	07/03/2026 21:20	JavaScript Source ...

Figura 1. Estructura de archivos utilizada para organizar el desarrollo del formulario.

4. Diseño del formulario HTML y validación básica con HTML

El formulario se ha construido en HTML con una estructura pensada para que los distintos campos queden claramente organizados y resulten fáciles de identificar. Se incluyen datos habituales en un registro de este tipo, como nombre de usuario, nombre, apellido, fecha de nacimiento, teléfono, contraseña y correo electrónico, además de los campos de repetición para correo y contraseña.

A nivel de maquetación, se ha buscado una distribución limpia y centrada, utilizando etiquetas visibles, textos de ayuda dentro de los campos y una separación clara entre bloques. Algunos elementos, como el nombre y el apellido o el código de país y el número de teléfono, se muestran agrupados en la misma línea para aprovechar mejor el espacio y mejorar la lectura visual del formulario.

En esta parte también se ha aprovechado la validación básica que ofrece HTML5. Por ejemplo, se han utilizado atributos como *required* para obligar a completar ciertos campos, *type="email"* para el correo electrónico y *type="date"* para la fecha de nacimiento. Estos recursos permiten incorporar una primera capa de control antes incluso de aplicar la lógica adicional programada en JavaScript.

Aunque la validación principal se desarrolla posteriormente desde JavaScript, el uso de HTML5 sigue siendo útil como apoyo, ya que mejora la semántica del formulario y proporciona una base más sólida desde el propio navegador.

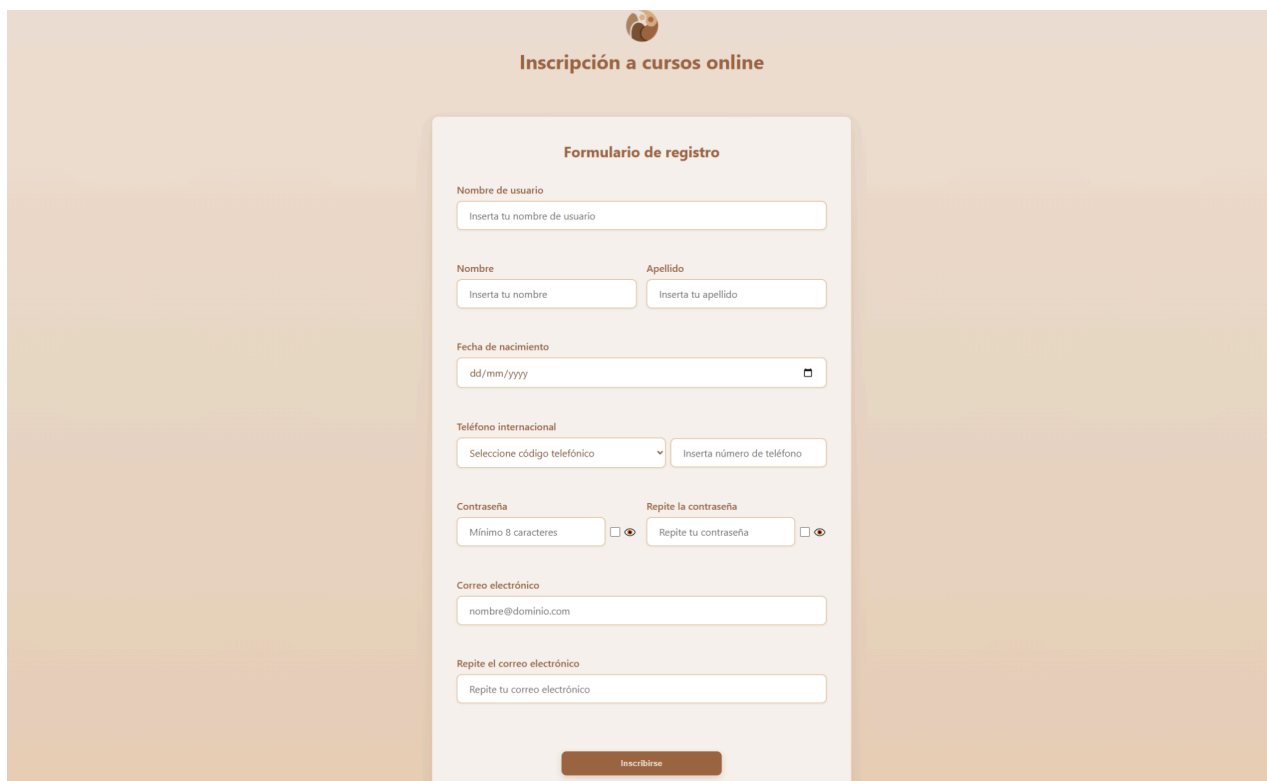
The image shows a web registration form titled "Inscripción a cursos online" with a sub-header "Formulario de registro". The form is centered on a light orange background. It contains several input fields: "Nombre de usuario" (with placeholder "Inserta tu nombre de usuario"), "Nombre" and "Apellido" (with placeholders "Inserta tu nombre" and "Inserta tu apellido" respectively), "Fecha de nacimiento" (with placeholder "dd/mm/yyyy" and a calendar icon), "Teléfono internacional" (with a dropdown for "Seleccione código telefónico" and an input for "Inserta número de teléfono"), "Contraseña" (with placeholder "Mínimo 8 caracteres" and a toggle for visibility), "Repite la contraseña" (with placeholder "Repite tu contraseña" and a toggle for visibility), "Correo electrónico" (with placeholder "nombre@dominio.com"), and "Repite el correo electrónico" (with placeholder "Repite tu correo electrónico"). A brown "Inscribirse" button is at the bottom.

Figura 2. Vista general del formulario de inscripción desarrollado en HTML y estilizado con CSS.

```

        <div class="fila1">
            <label for="usuario">Nombre de usuario</label>
            <input type="text" id="usuario" name="usuario" placeholder="Inserta tu nombre
de usuario" required>
            <span id="error-usuario" class="error" aria-live="polite"></span>
        </div>

        <div class="fila2">
            <div>
                <label for="nombre">Nombre</label>
                <input type="text" id="nombre" name="nombre" placeholder="Inserta tu
nombre" required>
                <span id="error-nombre" class="error" aria-live="polite"></span>
            </div>
            <div>
                <label for="apellido">Apellido</label>
                <input type="text" id="apellido" name="apellido" placeholder="Inserta tu
apellido" required>
                <span id="error-apellido" class="error" aria-live="polite"></span>
            </div>
        </div>

        <div class="fila1">
            <label for="fecha-nacimiento">Fecha de nacimiento</label>
            <input type="date" id="fecha-nacimiento" name="fecha-nacimiento" required>
            <span id="error-fecha-nacimiento" class="error" aria-live="polite"></span>
        </div>

```

Fragmento 1. Parte de la estructura HTML del formulario, con campos obligatorios, agrupación visual y contenedores para mensajes de error.

5. Validación con JavaScript y expresiones regulares

Aunque HTML5 aporta una primera capa de validación, la mayor parte del control se ha desarrollado en JavaScript para poder comprobar los datos con más precisión y adaptar mejor la respuesta del formulario a cada caso. De esta forma, no solo se detecta si un campo está vacío o tiene un formato básico incorrecto, sino que también se pueden aplicar reglas más concretas según el tipo de dato introducido.

Para organizar esta parte, se han separado las funciones de validación en el archivo *validadores.js*, dejando en él las comprobaciones reutilizables de campos como el nombre de usuario, el nombre y apellido, el correo electrónico, la contraseña, el teléfono o la coincidencia entre valores repetidos. Esto permite evitar código duplicado y hace que la lógica resulte más clara y más fácil de probar por separado.

Las expresiones regulares se han utilizado especialmente en aquellos campos donde el formato es importante, como ocurre con el correo electrónico, la contraseña o el teléfono. En el caso de la contraseña, por ejemplo, no basta con comprobar una longitud mínima, sino que también se exige la presencia de mayúsculas, minúsculas, números y caracteres especiales. En el correo electrónico, la validación comprueba que la estructura general sea correcta, mientras que en el teléfono se adapta la comprobación según el prefijo seleccionado.

Además, en algunos casos no solo se valida el formato, sino también la relación entre campos. Esto ocurre al repetir el correo electrónico o la contraseña, donde se comprueba que ambos valores coincidan. También se ha incluido una verificación simulada de duplicidad para nombre de usuario, correo y teléfono, utilizando datos locales de ejemplo para aproximarse al comportamiento que tendría una validación conectada a un sistema real.

Gracias a esta organización, la validación no depende únicamente de las restricciones básicas del navegador, sino que incorpora una lógica más completa y controlada desde el propio código, permitiendo un comportamiento más preciso y una mejor experiencia de uso.

```

<div class="fila1">
  <label for="telefono">Teléfono internacional</label>
  <div class="campo-telefono">
    <select id="codigo-pais" name="codigo-pais" aria-label="Código de país">
      <option value="" selected>Seleccione código telefónico</option>
      <option value="+34">+34 (España)</option>
      <option value="+49">+49 (Alemania)</option>
      [...]
      <option value="+58">+58 (Venezuela)</option>
      <option value="+84">+84 (Vietnam)</option>
    </select>
    <input type="tel" id="telefono" name="telefono" placeholder="Inserta número de teléfono"
required>
  </div>
  <span id="error-telefono" class="error" aria-live="polite"></span>
</div>
<div class="fila2">
  <div>
    <label for="contrasena">Contraseña</label>
    <div class="campo-pass">
      <input
        type="password"
        id="contrasena"
        name="contrasena"
        placeholder="Mínimo 8 caracteres"
        minlength="8"
        pattern="(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^\w\s]).{8,}"
        title="La contraseña debe tener al menos 8 caracteres e incluir una letra
minúscula, una mayúscula, un número y un carácter especial."
        required>
      <label class="ojo" for="ver-contrasena" aria-label="Mostrar u ocultar contraseña">
        <input type="checkbox" id="ver-contrasena">
        
      </label>
    </div>
    <span id="error-contrasena" class="error" aria-live="polite"></span>
  </div>
  <div>
    <label for="contrasena2">Repite la contraseña</label>
    <div class="campo-pass">
      <input
        type="password"
        id="contrasena2"
        name="contrasena2"
        placeholder="Repite tu contraseña"
        required>
      <label class="ojo" for="ver-contrasena2" aria-label="Mostrar u ocultar repetición de
contraseña">
        <input type="checkbox" id="ver-contrasena2">
        
      </label>
    </div>
    <span id="error-contrasena2" class="error" aria-live="polite"></span>
  </div>
</div>
<div class="fila1">
  <label for="email">Correo electrónico</label>
  <input type="email" id="email" name="email" placeholder="nombre@dominio.com" required>
  <span id="error-email" class="error" aria-live="polite"></span>
</div>
<div class="fila1">
  <label for="email2">Repite el correo electrónico</label>
  <input type="email" id="email2" name="email2" placeholder="Repite tu correo electrónico"
required>
  <span id="error-email2" class="error" aria-live="polite"></span>
</div>

```

Fragmento 2. Campos de contraseña y correo con validación básica en HTML y apoyo para validación posterior en JavaScript.

En el siguiente fragmento puede verse parte de la lógica de validación implementada en JavaScript, donde se combinan expresiones regulares y comprobaciones específicas según el tipo de dato.

```
const PATRONES = {
  usuario: /^[a-zA-Z0-9_]+$/,
  palabraPersona: /^[A-ZÁÉÍÓÚŨ][a-záéíóúũñ]+(?:[A-ZÁÉÍÓÚŨ][a-záéíóúũñ]+)*$/,
  nombreCompleto: /^[A-Za-zÁÉÍÓÚŨáéíóúũñ]+(?:\s+[A-Za-zÁÉÍÓÚŨáéíóúũñ]+)+$/,
  correo: /^[^\s@]+@[^\s@]+\.[^\s@]+$/,
  telefonoInterno: /^[0-9\s-]+$/,
  contrasena: /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^\w\s]).{8,}$/
};

/**
 * Comprueba si un email tiene formato válido.
 * @param {string} value Correo introducido por la persona usuaria.
 * @returns {boolean} `true` si cumple el formato de email.
 */
function esCorreoValido(value) {
  return PATRONES.correo.test(String(value).trim());
}

[...]
```

```
/**
 * Valida un teléfono en función del prefijo internacional seleccionado.
 * - Si el prefijo es +34, aplica reglas españolas (inicio 6/7/8/9 y 9 dígitos).
 * - Para otros prefijos, aplica validación genérica de 9 o 10 dígitos.
 * @param {string} codigoPais Prefijo internacional (por ejemplo, +34).
 * @param {string} telefono Número nacional sin prefijo.
 * @returns {boolean} `true` si el número cumple la regla correspondiente.
 */
function esTelefonoPorCodigo(codigoPais, telefono) {
  const codigo = String(codigoPais || '').trim();
  const valor = String(telefono || '').trim();

  if (!codigo || !valor || !PATRONES.telefonoInterno.test(valor)) {
    return false;
  }

  const telefonoNormalizado = normalizarTelefono(valor);

  if (codigo === '+34') {
    return /^[6789]\d{8}$/.test(telefonoNormalizado);
  }

  return /^[d{9,10}$/.test(telefonoNormalizado);
}

[...]
```

```
/**
 * Comprueba la fortaleza de una contraseña.
 * Requisitos: 8+ caracteres, mayúscula, minúscula, número y carácter especial.
 * @param {string} value Contraseña introducida por la persona usuaria.
 * @returns {boolean} `true` si cumple todos los requisitos de seguridad.
 */
function esContrasenaValida(value) {
  return PATRONES.contrasena.test(String(value));
}

[...]
```

```
/**
 * Comprueba si dos valores son iguales tras normalizar espacios laterales.
 * @param {string} izquierda Primer valor.
 * @param {string} derecha Segundo valor.
 * @returns {boolean} `true` si ambos valores coinciden.
 */
function sonValoresIguales(izquierda, derecha) {
  return String(izquierda).trim() === String(derecha).trim();
}
```

Fragmento 3. Ejemplo de validación en JavaScript para correo, teléfono, contraseña y coincidencia entre valores.

6. Gestión de eventos y experiencia de usuario

Además de validar los campos, también se ha trabajado la interacción con la persona usuaria para que el formulario resulte más claro y fácil de utilizar. Para ello, se han gestionado distintos eventos de JavaScript que permiten reaccionar en el momento en que la persona escribe, entra en un campo, sale de él o intenta enviar el formulario.

El evento *input* se utiliza para realizar comprobaciones en tiempo real, mostrando mensajes de error o confirmación a medida que cambian los valores introducidos. Esto permite detectar fallos de forma inmediata, sin necesidad de esperar al envío final, y facilita que la persona usuaria pueda corregirlos sobre la marcha.

Los eventos *focus* y *blur* ayudan a reforzar visualmente la interacción. Cuando un campo recibe el foco, se destaca para indicar de forma más clara dónde se está escribiendo. Al salir del campo, se revisa su contenido y se actualiza el estado visual correspondiente, de modo que el formulario no solo informa mediante texto, sino también a través de cambios de estilo.

Por otra parte, el evento *submit* permite interceptar el envío del formulario para comprobar que todos los datos sean correctos antes de continuar. Si existe algún error, el envío se bloquea y se muestran los avisos necesarios. Si todo es válido, se simula un envío asíncrono y se presenta un mensaje de confirmación. Además, se limpian los datos guardados temporalmente y el formulario vuelve a su estado inicial.

Todo esto hace que la interacción resulte más fluida y comprensible, ya que la validación no aparece únicamente al final, sino que acompaña a la persona usuaria durante todo el proceso de cumplimentación.



Formulario de registro

Nombre de usuario

d

Debe tener al menos 5 caracteres y solo letras, números, "_" o ".".

Nombre

Apellido

Inserta tu nombre

Inserta tu apellido

Este campo es obligatorio.

Este campo es obligatorio.

Fecha de nacimiento

07/03/2026

Debes ser mayor de 18 años para registrarte.

Figura 3. Ejemplo de validación en tiempo real en varios campos del formulario.

The image shows a registration form with several fields and error messages:

- Teléfono internacional:** A dropdown menu shows "+34 (España)" and a text input shows "547484". Below it, a message says: "El número debe tener 9 dígitos y empezar por 6, 7, 8 o 9."
- Contraseña:** A password input field with a strength indicator (a small eye icon). Below it, a message says: "Mínimo 8 caracteres con mayúscula, minúscula, número y símbolo."
- Repite la contraseña:** A second password input field. Below it, a message says: "Las contraseñas no coinciden."
- Correo electrónico:** A text input field showing "sdfsdfsdf". Below it, a message says: "El correo no tiene un formato válido."
- Repite el correo electrónico:** A text input field with a placeholder "Repite tu correo electrónico". Below it, a message says: "Debes repetir el correo electrónico."
- Inscribirse:** A large orange button at the bottom.

Figura 4. Diferentes mensajes de error mostrados en tiempo real según el tipo de validación aplicada.

7. Pruebas unitarias con Jest y documentación con JSDocs

Además de la validación aplicada en el navegador, también se ha trabajado la comprobación del código desde una perspectiva más técnica mediante pruebas unitarias con Jest y documentación con JSDoc. Esto permite no solo verificar que las funciones se comporten como se espera, sino también describir su finalidad y facilitar la comprensión del código.

Las pruebas unitarias se han desarrollado en el archivo *validadores.test.js*, donde se comprueban distintos casos válidos e inválidos para las funciones incluidas en *validadores.js*. De esta forma, no se prueba únicamente el comportamiento visual del formulario, sino también la lógica interna que se encarga de validar cada dato. Entre los casos comprobados se incluyen, por ejemplo, nombres de usuario incorrectos, correos con formato no válido, contraseñas que no cumplen los requisitos, teléfonos erróneos, fechas de nacimiento que no alcanzan la mayoría de edad y comparaciones entre campos repetidos.

Este tipo de pruebas resulta útil porque permite detectar errores con más seguridad y comprobar que las funciones responden correctamente ante situaciones diferentes, incluidas algunas menos evidentes. Además, al estar separadas del navegador, ayudan a revisar la lógica de validación de forma más directa y ordenada.

Por otra parte, se han incorporado comentarios JSDoc en las funciones principales del archivo *validadores.js*, indicando qué hace cada una, qué parámetros recibe y qué valor devuelve. A partir de estos comentarios se ha generado una documentación en la

carpeta *docs*, lo que permite consultar de forma más clara la estructura y responsabilidad de las funciones reutilizables.

En conjunto, tanto las pruebas con Jest como la documentación con JSDoc aportan una capa adicional de calidad al trabajo realizado, ya que no solo se busca que el formulario funcione, sino también que el código quede mejor comprobado, mejor descrito y más fácil de mantener.

```
PS E:\GIT\DAW\2nd-year\desarrollo-web-entorno-cliente\ud5-validacion-de-formularios-web-y-documentacion-de-codigo-con-javascript> npm test
PASS ./validadores.test.js
  Utilidades
    ✓ normaliza teléfono eliminando espacios y guiones (2 ms)
  Compatibilidad de exportación del módulo
    ✓ expone validadores en globalThis cuando no existe module.exports (2 ms)
    ✓ expone validadores en globalThis cuando no existe window (1 ms)
  Validación de usuario
    ✓ acepta usuario válido (1 ms)
    ✓ acepta usuario válido con espacios laterales (trim) (1 ms)
    ✓ rechaza usuario corto
    ✓ rechaza usuario con espacios
    ✓ rechaza usuario con caracteres no permitidos
  Validación de nombre y apellido
    ✓ acepta palabra con inicial mayúscula
    ✓ acepta palabra con tildes y ñ (1 ms)
    ✓ acepta nombre compuesto con ambas iniciales en mayúscula
    ✓ rechaza palabra en minúsculas
    ✓ rechaza palabra con guion
  Validación de nombre completo
    ✓ acepta nombre y apellido válidos (1 ms)
    ✓ acepta nombre completo con espacios laterales
    ✓ rechaza un solo nombre
    ✓ rechaza números en el nombre
    ✓ rechaza separadores no alfabéticos
  Validación de correo electrónico
    ✓ acepta un correo válido
    ✓ acepta correo válido con espacios laterales
    ✓ rechaza correo inválido
    ✓ rechaza correo sin dominio de nivel superior
    ✓ rechaza correo con espacios internos
  Validación de teléfono internacional
    ✓ rechaza teléfono cuando no se envía valor
    ✓ acepta el formato completo +34 600-123-456 (1 ms)
    ✓ acepta +34 sin espacio tras prefijo
    ✓ rechaza teléfono internacional con prefijo distinto en esTelefonoValido (1 ms)
    ✓ con +34 acepta formato sin separadores internos
    ✓ con +34 acepta formato con espacios internos
    ✓ con +34 acepta móviles que empiezan por 7
    ✓ con +34 acepta fijos que empiezan por 9
    ✓ con +34 rechaza prefijos de numeración española no válidos
    ✓ con +34 rechaza longitud distinta de 9 dígitos
    ✓ para otros prefijos acepta 9 dígitos
    ✓ rechaza teléfono si no se selecciona código de país
    ✓ para otros prefijos acepta 10 dígitos
    ✓ para otros prefijos rechaza menos de 9 dígitos
    ✓ para otros prefijos rechaza más de 10 dígitos
    ✓ rechaza teléfono vacío
    ✓ rechaza separadores no permitidos
  Validación de contraseña segura
    ✓ acepta contraseña con todos los requisitos
    ✓ acepta contraseña válida de 8 caracteres exactos
    ✓ rechaza contraseña sin carácter especial
    ✓ rechaza contraseña sin mayúscula
    ✓ rechaza contraseña sin minúscula
    ✓ rechaza contraseña sin número
    ✓ rechaza contraseña de menos de 8 caracteres
  Validación de mayoría de edad
    ✓ acepta fecha de nacimiento con 18 o más años (1 ms)
    ✓ acepta exactamente 18 años cumplidos hoy
    ✓ rechaza fecha de nacimiento menor de edad
    ✓ rechaza fecha vacía
    ✓ rechaza fecha inválida
    ✓ rechaza si cumple 18 mañana (aún menor)
  Validación de coincidencia entre campos
    ✓ acepta valores iguales
    ✓ acepta valores iguales con espacios laterales
    ✓ rechaza valores diferentes
    ✓ rechaza por diferencia de mayúsculas/minúsculas (1 ms)

Test Suites: 1 passed, 1 total
Tests:       57 passed, 57 total
Snapshots:   0 total
Time:        0.701 s, estimated 1 s
Ran all test suites.
```

Figura 5. Ejecución de las pruebas unitarias con Jest y resultado satisfactorio de todos los casos comprobados.

```
PASS ./validadores.test.js
[...]
```

```
Test Suites: 1 passed, 1 total
Tests:      57 passed, 57 total
Snapshots:  0 total
Time:       0.701 s, estimated 1 s
Ran all test suites.
```

Fragmento 4. Resumen de las pruebas, extraído de la salida en terminal para facilitar la legibilidad

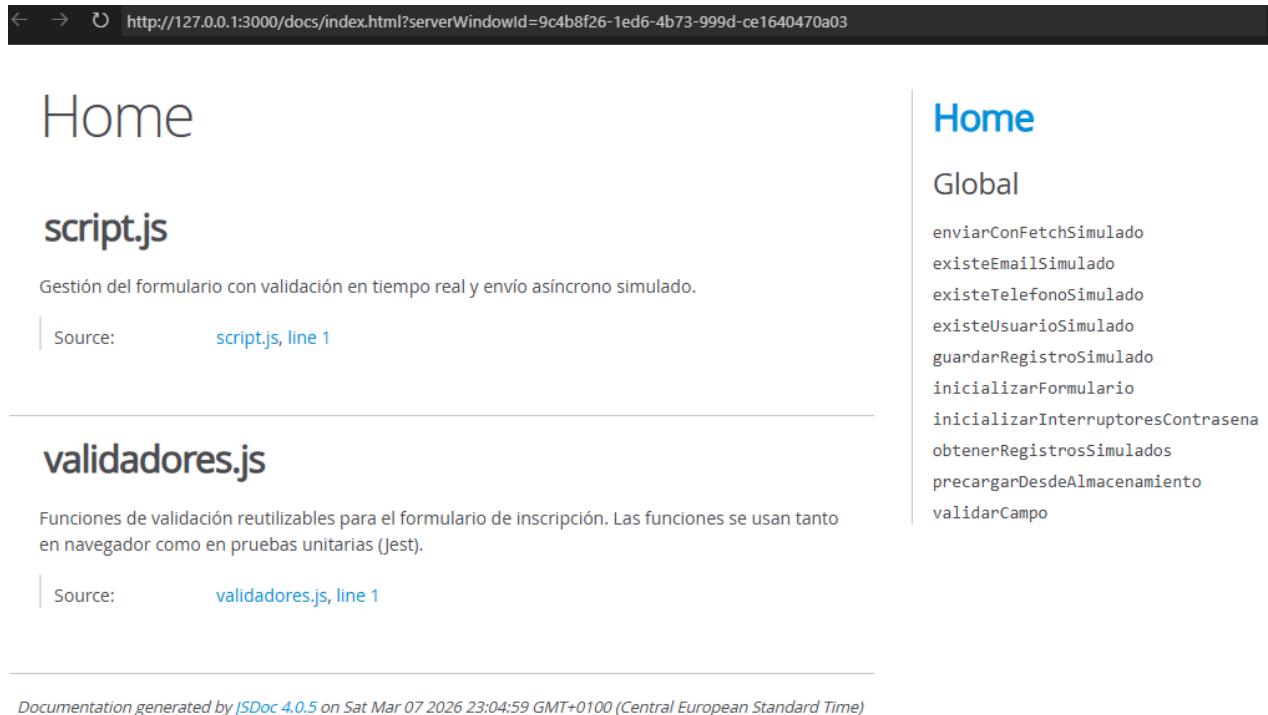


Figura 6. Documentación generada automáticamente con JSDoc a partir de los comentarios incluidos en el código.

8. Conclusión

El desarrollo de este trabajo me ha permitido aplicar de forma práctica varios de los contenidos propios del desarrollo web en entorno cliente, no solo desde la parte visual del formulario, sino también desde la validación, la interacción y la organización del código. A lo largo del proceso he trabajado con HTML, CSS y JavaScript de forma combinada, utilizando cada tecnología para una función distinta dentro del resultado final.

Uno de los aspectos más importantes ha sido comprobar que validar no consiste solo en impedir el envío de datos incorrectos, sino también en guiar a la persona usuaria mientras completa el formulario. La validación en tiempo real, los mensajes de error y los cambios visuales ayudan a que la interacción resulte más clara y más cómoda, evitando que todos los fallos aparezcan únicamente al final.

También ha resultado útil separar la lógica de validación en funciones reutilizables, ya que esto hace que el código sea más ordenado, más fácil de entender y más sencillo de comprobar mediante pruebas unitarias. Del mismo modo, la documentación con JSDoc aporta una mejora a nivel de mantenimiento, al dejar descrita la finalidad de cada función y su uso principal.

Además, esta práctica también me ha resultado útil de cara al proyecto final, ya que me ha permitido reforzar y ordenar mejor aspectos que ya forman parte de su desarrollo, especialmente en la validación de formularios, la organización de la lógica en JavaScript y la forma de gestionar la interacción con la persona usuaria. Aunque el proyecto final tiene un alcance mayor y requiere otras capas adicionales, este trabajo me ha servido para afianzar una parte importante del lado cliente y aplicar de forma más consciente varias decisiones de implementación.

En definitiva, este trabajo no solo me ha servido para construir un formulario funcional, sino también para reforzar la importancia de organizar mejor la lógica, comprobar el comportamiento del código y cuidar más la parte de interacción con la persona usuaria. Es una práctica bastante completa porque no se queda solo en que el formulario “funcione”, sino en cómo responde y cómo está construido.

9. Referencias

- **Materiales de la unidad UD5:** *Interacción con el usuario: Eventos y formularios.*
- **Mozilla Developer Network (MDN):**
<https://developer.mozilla.org/>
Consultado para revisar el uso de validación básica en HTML5, tipos de entrada como correo electrónico o fecha, así como apoyo en la validación mediante expresiones regulares.
- **W3Schools:**
<https://www.w3schools.com/js/>
Consultado como apoyo complementario para revisar ejemplos prácticos de validación de formularios y gestión de eventos en el navegador.
- **Mimo:**
Utilizada como apoyo complementario para reforzar conceptos relacionados con JavaScript, validación de datos y organización de la lógica en el lado cliente.
- **YouTube:** *Getting Started Testing with Jest: Part 1.*
<https://www.youtube.com/watch?v=HgXtmAG5H6M>
Utilizado como apoyo para comprender la instalación inicial de Jest, la estructura de las pruebas unitarias y la ejecución de tests en un proyecto JavaScript.
- **YouTube:** *Getting Started Testing with Jest: Part 2.*
<https://www.youtube.com/watch?v=lx9o7cRd62s>
Utilizado como continuación del recurso anterior para reforzar el uso de Jest y entender mejor la organización y ampliación de las pruebas.
- **YouTube:** *Documenting Your JavaScript | JSDoc Crash Course.*
<https://youtu.be/YK-GurROGIq?si=BQRQgh6hydZzhZ6l>
Utilizado como apoyo para comprender el uso de comentarios JSDoc y la generación automática de documentación a partir del código fuente.